



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA INGENIERÍA SOFTWARE

ASIGNATURA:

APLICACIONES WEB

NOMBRE:

IRVIN MARCELO CAJAS IBARRA

FECHA:

LUNES, 15 DE JUNIO DEL 2026

(Semana VII)

1 Introducción

En el desarrollo de software es habitual que una aplicación evolucione con el tiempo, incorporando nuevas funcionalidades sobre lo que ya existe. Esta práctica parte de un proyecto ASP.NET Core MVC previamente desarrollado y lo extiende agregando una tabla mejorada para visualizar los datos, la capacidad de eliminar registros con confirmación, y validación en el lado del cliente para mejorar la experiencia del usuario.

2 Paso 1 — Tabla HTML con lista de productos

2.1 Código de la Vista Index actualizada

```
@model IEnumerable<ProyectoMVC.Models.Producto>

<h2>Catálogo de Libros</h2>

<a asp-action="Create" class="btn btn-primary mb-3">+ Agregar Libro</a>

<table class="table table-bordered table-hover">
  <thead class="table-dark">
    <tr>
      <th>#</th>
      <th>Título</th>
      <th>Autor</th>
      <th>Precio</th>
      <th>Páginas</th>
      <th>ISBN</th>
      <th>Acciones</th>
    </tr>
  </thead>
  <tbody>
    @if (!Model.Any())
    {
      <tr>
        <td colspan="7" class="text-center text-muted">No hay
libros registrados.</td>
      </tr>
    }
    @foreach (var libro in Model)
    {
      <tr>
        <td>@libro.Id</td>
        <td>@libro.Título</td>
        <td>@libro.Autor</td>
```

```

                <td>${@libro.Precio}</td>
                <td>@libro.Paginas</td>
                <td>@libro.ISBN</td>
                <td>
                    <form asp-action="Delete" asp-route-id="@libro.Id"
method="post">
                        @Html.AntiForgeryToken()
                        <button type="submit" class="btn btn-danger btn-sm"
onclick="return confirm('¿Seguro que deseas
eliminar este libro?')">
                            Eliminar
                        </button>
                    </form>
                </td>
            </tr>
        }
    </tbody>
</table>

```

2.2 Vista Index en el navegador

Se actualizó la vista Index con una tabla mejorada que incluye encabezado oscuro, columna de número de fila y columna de Acciones con el botón Eliminar en cada registro.

2.3 Tabla vacía con mensaje

Cuando no existe ningún libro en la lista, la tabla muestra un mensaje centrado indicando que no hay libros registrados, en lugar de aparecer vacía sin ninguna indicación al usuario.

3 Paso 2 — Acción Delete con confirmación POST y CSRF

3.1 Código del controlador con la acción Delete

```

using Microsoft.AspNetCore.Mvc;
using ProyectoMCV.Models;

namespace ProyectoMCV.Controllers
{
    public class ProductoController : Controller
    {
        private static List<Producto> _productos = new List<Producto>();

        public IActionResult Index()

```

```

    {
        return View(_productos);
    }

    public IActionResult Create()
    {
        return View();
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Create(Producto producto)
    {
        if (ModelState.IsValid)
        {
            producto.Id = _productos.Count + 1;
            _productos.Add(producto);
            return RedirectToAction("Index");
        }
        return View(producto);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult Delete(int id)
    {
        var producto = _productos.FirstOrDefault(p => p.Id == id);
        if (producto != null)
        {
            _productos.Remove(producto);
        }
        return RedirectToAction("Index");
    }
}

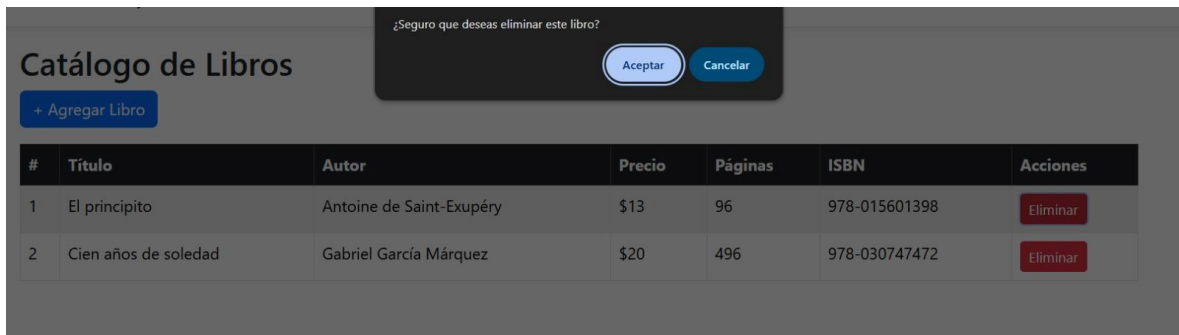
```

3.2 Actualización del controlador en el editor

Se actualizó el controlador agregando la acción Delete con los atributos [HttpPost] y [ValidateAntiForgeryToken], garantizando que la eliminación solo pueda realizarse mediante una solicitud POST legítima con el token de seguridad válido.

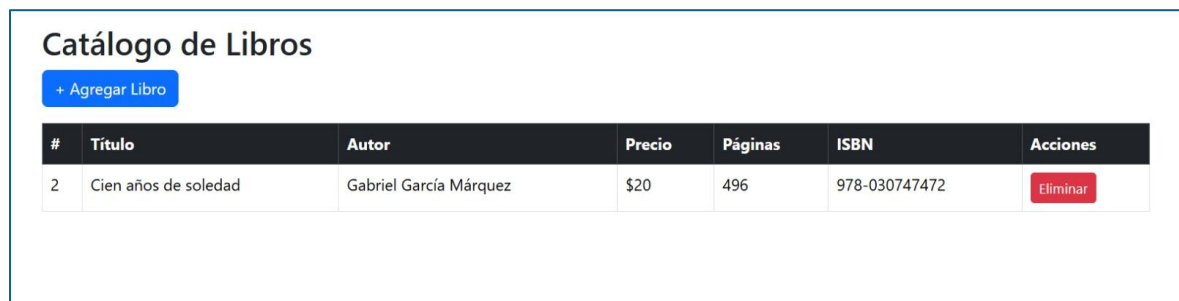
3.3 Confirmación de eliminación

Al presionar el botón Eliminar en cualquier fila de la tabla, el navegador muestra un cuadro de diálogo pidiendo confirmación antes de proceder con la eliminación del registro.



3.4 Libro eliminado exitosamente

Tras confirmar la eliminación, el controlador remueve el libro de la lista y redirige a la vista Index, donde se puede verificar que el registro ya no aparece en la tabla.



4 Paso 3 — Validación cliente y servidor

4.1 Actualización de la Vista Create con validación cliente

```
@model ProyectoMCV.Models.Producto

<h2>Agregar Libro</h2>

<form asp-action="Create" method="post">
    @Html.AntiForgeryToken()

    <div class="mb-3">
        <label asp-for="Titulo" class="form-label"></label>
        <input asp-for="Titulo" class="form-control" />
        <span asp-validation-for="Titulo" class="text-danger"></span>
    </div>

    <div class="mb-3">
```

```

        <label asp-for="Autor" class="form-label"></label>
        <input asp-for="Autor" class="form-control" />
        <span asp-validation-for="Autor" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Precio" class="form-label"></label>
        <input asp-for="Precio" class="form-control" />
        <span asp-validation-for="Precio" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Paginas" class="form-label"></label>
        <input asp-for="Paginas" class="form-control" />
        <span asp-validation-for="Paginas" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="ISBN" class="form-label"></label>
        <input asp-for="ISBN" class="form-control" />
        <span asp-validation-for="ISBN" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-success">Guardar</button>
    <a asp-action="Index" class="btn btn-secondary">Cancelar</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

4.2 Validación cliente en el navegador

Se actualizó la vista Create agregando el bloque `@section Scripts` con `_ValidationScriptsPartial`, activando jQuery Validation para mostrar mensajes de error directamente en el navegador sin necesidad de recargar la página.

Agregar Libro

Título

El título es obligatorio

Autor

El autor es obligatorio

Precio

El precio es obligatorio

Páginas

El número de páginas es obligatorio

ISBN

El ISBN es obligatorio

Guardar

Cancelar

4.3 Validación servidor en el controlador

En caso de que la validación del cliente sea omitida, el servidor verifica los datos mediante `ModelState.IsValid`. Si algún campo no cumple las reglas del modelo, se devuelve el formulario con los mensajes de error correspondientes.

Agregar Libro

Título

El título es obligatorio

Autor

El autor es obligatorio

Precio

The value " " is invalid.

Páginas

The value " " is invalid.

ISBN

El ISBN es obligatorio

Guardar

Cancelar

5 Paso 5 — README con instrucciones de ejecución

5.1 README en el repositorio

Se añadió el archivo README.md en la raíz del repositorio con las instrucciones necesarias para clonar y ejecutar el proyecto, incluyendo los requisitos previos y los pasos detallados para ponerlo en marcha.

6 Repositorio GitHub

Link: <https://github.com/Theirvin1/GS--Proyecto-ASP.NET-Core-MVC>

7 Conclusión

Esta extensión permitió profundizar en aspectos importantes del desarrollo web con ASP.NET Core, como la diferencia entre validación cliente y servidor, donde la primera mejora la experiencia del usuario al mostrar errores sin recargar la página, y la segunda garantiza la integridad de los datos en el backend. El uso de ramas en Git demostró ser una práctica útil para separar el trabajo nuevo sin afectar lo que ya funcionaba.